

Tutorial 03

▼ Overview

Tutorial

- Sorting Algorithms
- Revision Questions at the Bottom

Lab

- Practical Aspect of Computational Complexity (Covered in Lab)
- Systematic Approach to Problem Solving (In Tut and Lab)
- Scientific Reasoning and Conclusions (Covered in Lab)
- Analysis Skills (In Tut and Lab)
- Algorithms Identification (In Tut and Lab)

This weeks lab is entirely hand marked.

Please ensure you read the bottom of the lab page, although the scaffold covers a lot it is not sufficient for this task.

▼ Visualiser

Visualiser is from <https://visualgo.net/en/sorting> know as VisuAlgo's

- ▼ What sorting algorithms have we encountered?
 - ▼ What is Bubble Sort with early exit?
 - Swaps neighbouring elements that are in the wrong order.
 - (This only happens with early exit) Once a pass is completed where no elements are swapped the array is deemed sorted.
 - Time complexity is $O(n^2)$.
 - Has to do at most $n-1 + n-2 + \dots + 1 = n * (n-1) / 2$ swaps each pass (as the largest element of the current sort will be moved all the way to the end each pass).
 - Adaptable so when ordered time complexity is $O(n)$.
 - Stable sort as the swaps are not long range and not swapping like elements.

[attachment:fa5dc24d-44ab-4c15-af39-9a96ce0f0cf9:2025-09-30_00-29-24.mov](#)

▼ What is Selection Sort?

- Selects the lowest element in the current array, moves it to the start.
- Time Complexity $O(n^2)$. This requires $n-1$ comparisons the first pass, $n-2$ the next so on so the time complexity is $n-1 + \dots + 1 =$ approximately $n * (n-1) / 2$ due to triangular numbers series which is $O(n^2)$,
- Non- adaptive
- Non-stable (long range swaps).

[attachment:8602b4bc-8a24-4a51-822f-2e26cbb6658e:2025-09-30_00-30-27.mov](#)

▼ What is Insertion Sort?

- Takes the next element and inserts it ordered into a subarray of the previous elements. By subarray I mean all elements before that current element in the array which is sorted.
- $O(n^2)$ time complexity for similar reasons to bubble and selection sorts.
- Stable
- Adaptive (sorted takes $O(n)$ time)

[attachment:674145c0-ffd2-48c7-90de-1bc266f7d89d:2025-09-30_00-35-17.mov](#)

▼ What is Merge Sort?

1. Divides the array first into n elements recursively this is a $\log(n)$ division potentially dividing the array each time.
2. Merges the arrays back together going through n potential elements each time as we are iterating through each array at the same time.

- a. The reason we can go through each array together, they should both be sorted in ascending order from the last merge operation. For example if we divide to singular elements individual elements are ordered, then groups of two elements are ordered, then 4. $\log(n)$ times merging halving the amount the function is run each time.
 - b. Copying the arrays back to the original array is n for each level meaning $n * \log(n)$.
- Time Complexity is $O(n * \log n)$ we run the algorithm by combining two halves $O(n * \log n)$ each and add the time taken to combine the arrays. $O(n) < O(n * \log n)$. Or using above $O(n * \log n)$ is the largest operation.
 - Non-adaptive, always the same
 - Stable comparisons can prioritise the first element,
 - Not in place, needs to create an extra array to copy the sorting of either side of the division.

[attachment:852faa9c-9eac-440f-afd8-84c3e8175a65:2025-09-30_00-35-59.mov](#)

▼ What is Quick Sort?

1. Takes a partition, swapping about the partition point to get to a midpoint from the right and left where all on the right are greater and all on the left are less.
 2. Then it does the same to each side left and right of the partition.
- $O(n * \log n)$ average Time Complexity
 - Needs to go through n elements on either side of the partition each time $\log n$ times as we are splitting about a partition approximately in the middle
 - Unless picking a bad pivot. Worst case is $O(n^2)$ picking a bad pivot and now having to go through e.g. only the left or right each time.
 - Not adaptive - although the average is $O(n * \log n)$ adding different ordered inputs does not change this.
 - Not stable (long range swaps).
 - Choosing the partition makes it more likely to get the average case hence we sometimes use

- Normally (Choose the first element - Think of cases this would be worse for)
- Median of Three (Compare the middle, start and end and then choose the number in the middle.)
- Randomised (Choose a random element)

[attachment:584883a7-a66b-49ff-82a8-de44d3250139:2025-09-30_00-36-47.mov](#)

[attachment:8f7f68cd-d96f-48c2-8488-7aec9edd3cfb:2025-09-30_00-37-31.mov](#)

▼ What is Radix Sort?

1. Choose a stable sort (preferred merge as it is quick too)
 2. Run it multiple times but over first the least important index to the most important.
 - a. Each time you put them in boxes / buckets based on the index they have to then retain the order that sort had at the end.
- E.g. words (sort letter), numbers (sort digits), students with zids (sort name (letters) and zid (digits)).

[attachment:79beafb5-b2b2-4a9b-ae6e-3d30e8e606c3:2025-09-30_00-38-16.mov](#)

▼ Visualiser

<https://visualgo.net/en/sorting>

▼ Manually Sort This Array By Name

Row Number	course	name	program
------------	--------	------	---------

1	COMP1927	Jane	3970
2	COMP1927	John	3978
3	COMP1927	Pete	3978
4	MATH1231	John	3978
5	MATH1231	Adam	3970
6	PSYC1011	Adam	3970
7	PSYC1011	Jane	3970

What order would the arrays row numbers be in if it was sorted stable.

▼ Answer

Row Number	course	name	program
5	MATH1231	Adam	3970
6	PSYC1011	Adam	3970
1	COMP1927	Jane	3970
7	PSYC1011	Jane	3970
2	COMP1927	John	3978
4	MATH1231	John	3978
3	COMP1927	Pete	3978

What is the order of row numbers using an unstable example? Give 2 examples.

▼ Answer (Potential)

The first is the stable.

Row Number	course	name	program
6	PSYC1011	Adam	3970
5	MATH1231	Adam	3970
7	PSYC1011	Jane	3970
1	COMP1927	Jane	3970
4	MATH1231	John	3978
2	COMP1927	John	3978
3	COMP1927	Pete	3978

▼ How does bubble sort (with early exit) perform compared to insertion sort in these situations?

▼ What does early exit mean?

When the bubble sort is fully sorted (no swaps are made in a passover) the bubble sort exits and stops running.

▼ What is the performance sorted (e.g., [1,2,3,...,n])?

- Bubble sort is $O(n)$ exits early because it is sorted
- Insertion sort makes a comparison to the next element to the last element inserted and since it is always greater it is always inserted $O(n)$ time complexity once for each element.
 - E. g.
 1. Start with (1,2,3)
 2. Insert 1 (1) (2,3) No comparison as no array.
 3. 2 is greater than 1, hence 2 is inserted $O(1)$, (1,2) (3)
 4. 3 is greater than 2, hence 3 is inserted $O(1)$, (1,2,3).
 - Hence it is clear the comparison only needs to be made to the last element

▼ What is the performance reverse sorted e.g. [n,n-1,n-2,...,1]?

- Bubble sort needs $n-1$ passes.
 1. The first will switch the element n all the way to the last position but not change the order of the rest of the array just the position that subarray is in. Takes $n-1$ comparisons / swaps.
 2. This action continues $n-1$ more times for each element comparing 1 less time then the element we are at e.g. $n-1$ swaps and compares $n-2$ times.
 - a. As seen earlier the addition of $1 + \dots + (n-2) + (n-1) \rightarrow$ time complexity $O(n^2)$.
- Insertion sort does the opposite
 1. Taking 1 comparison to compare n to $n-1$, 2 comparisons to compare $n-2$ to n and $n-1$ and so on until $n-1$ comparisons for 1.

a. Hence it is also $1 + \dots + (n-2) + (n-1) \rightarrow O(n^2)$ due to triangular numbers.

▼ What is the performance swapping the first and last element $[n, 2, 3, \dots, n-1, 1]$

- Bubble sort
 1. Will move n to the end making $n-1$ comparisons and easily putting it into the right position in one pass
 2. But then will have to do $n-1$ passovers to move the 1 all the way back as $n-1$ would swap with 1, then $n-2$ and so on but the bubble sort needs to go all the way through the array to get to 1 each time.
 - This is $O(n^2)$ as before triangular numbers.
- Insertion
 1. Compares the first element once so 2 to $n \rightarrow$ one comparison, insert 2 before n .
 2. Then the second element all the way to $n-1$ is 2 comparisons each as compare the number to n and then compare to the last inserted element and insert in front of that element.
 3. The final number 1 will be compared to every other element, $n-1$ times.
 - This is approximately $3n$ comparisons hence $O(n)$ time complexity.
- Very important that it must be the front and back switched as you don't know the order a sort is sweeping, what if it sweeps the reverse direction and orders that way? If you only swap one you risk getting an incorrect result.
- Good to note when comparing the two time complexities of these sorts as they are very similar.

▼ Merge Sort. Run Merge(A, 0, 4, 9)

lo	mid	hi	Array	
0	4	9	{ 1, 4, 5, 6, 7, 2, 3, 4, 7, 9 }	
nitems	tmp	i	j	k
	{ }			

▼ Sorted array in the end

{ 1, 2, 3, 4, 4, 5, 6, 7, 7, 9 }

// Typical Merge Sort

void mergeSort(int A[], in

```

t lo, int hi) {
    if (lo >= hi) return;

    int mid = (lo + hi) /
2;
    mergeSort(A, lo, mid);
    mergeSort(A, mid + 1,
hi);
    merge(A, lo, mid, hi);
}

void merge(int A[], int l
o, int mid, int hi) {
    int nitems = hi - lo +
1;
    int *tmp = malloc(nite
ms * sizeof(int));

    int i = lo; // Segment
1
    int j = mid + 1;
    int k = 0;

    // scan both segments
into tmp
    while (i <= mid && j <
= hi) { // Segment 2
        if (A[i] <= A[j])
        {
            tmp[k++] = A[i
++];
        } else {
            tmp[k++] = A[j
++];
        }
    }

    // copy items from unf
inished segment
    while (i <= mid) tmp[k
++] = A[i++]; // Segment 3
    while (j <= hi) tmp[k

```

```

    ++] = A[j++];

    // copy items from tmp
    back to main array
    for (i = lo, k = 0; i
    <= hi; i++, k++) { // Segment 4
        A[i] = tmp[k];
    }
    free(tmp);
}

```

▼ Show how the call `partition(A, 0, 9)` would partition each of the arrays.

- Quicksort algorithm important operation
 - Partition
 - Takes an element called the pivot
 - Reorganises the elements in the subarray `A[lo..hi]`
 - All elements to the left of the pivot in the subarray are less than or equal to the pivot,
 - All elements to the right of the pivot in the subarray are greater than the pivot.

```

int partition(int A[], int lo, int hi) {
    // Takes the lowest as the pivot
    int pivot = A[lo];

    int l = lo + 1;
    int r = hi;
    while (true) {
        while (l < r && A[l] <= pivot) l++;
        while (l < r && A[r] >= pivot) r--;
        if (l == r) break;
        swap(A, l, r);
    }

    if (pivot < A[l]) l--;
}

```

```

    swap(A, lo, l);
    return l;
}

```

What is the return index for each array, and how does the array end up, go line by line.

pivot	l	r	array
			{ 5, 3, 9, 6, 4, 2, 9, 8, 1, 7 }

▼ { 5, 3, 9, 6, 4, 2, 9, 8, 1, 7 }

Changes	To
pivot	5
l	1
r	9
l	2
r	8
array	{ 5, 3, 1, 6, 4, 2, 9, 8, 9, 7 }
l	3
r	5
array	{ 5, 3, 1, 2, 4, 6, 9, 8, 9, 7 }
l	5
l	4
array	{ 4, 3, 1, 2, 5, 6, 9, 8, 9, 7 }

Return index = 4

pivot	l	r	array
			{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }

▼ { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }

Changes	To
pivot	0
l	1
r	9
r	1
l	0
array	{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }

Index 0

pivot	l	r	array
			{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }

▼ { 9, 8, 7, 6, 5, 4, 3, 2, 1, 0 }

Changes	To
pivot	9
l	1
r	9
l	9
array	{0, 8, 7, 6, 5, 4, 3, 2, 1, 9 }

Index 9

▼ Show how the radix sort would sort the following array of strings

▼ How does a radix sort work?

Performs multiple stable sorts.

Initial	Name	First	Second	Third
0	set			
1	how			
2	cup			
3	hob			
4	paw			
5	hat			
6	cob			
7	pot			

▼ Answer

Initial	Name	First	Second	Third
0	set	hob	hat	cob
1	how	cob	paw	cup
2	cup	cup	set	hat
3	hob	set	hob	hob

4	paw	hat	cob	how
5	hat	pot	pot	paw
6	cob	how	how	pot
7	pot	paw	cup	set

Revision Problems On Course Website